

CS 6888: Program Analysis and its Applications

Assignment 4: Static Analysis

Due: March 25th at 5 PM
Worth 10 points

1 Learning Objective

To consolidate your understanding of, and obtain hands-on experience with, **Static Analysis**, including:

- how it works
- its strengths and weaknesses

2 Individual Assignment

This is an individual assignment. You are not to share your setup, strategies, results, analysis, code, or any other aspect of the assignment with your classmates.

3 Preparation

3.1 Background

This assignment will use two static analysis tools:, `infer` and `cppcheck`, to test `tcc` and another modified version of `tccas`.

3.2 Practice

Before you start the assignment, you should familiarize yourself with `infer` and `cppcheck`.

- <https://cppcheck.sourceforge.io/>
- <https://fbinfer.com/docs/getting-started/>
- <https://fbinfer.com/docs/hello-world#hello-world-make>

You may also want to familiarize yourself with pointers and operators in `c`.

- Pointers
https://www.tutorialspoint.com/cprogramming/c_pointers.htm
Notably: In most of the operating systems, programs are not permitted to access memory at address 0 because that memory is reserved by the operating system. Dereferencing a pointer that points to 0, otherwise referred to as NULL, will cause a crash.
- Operators
https://www.tutorialspoint.com/cprogramming/c_operators.htm

4 Tasks

4.1 Run the Tools

4.1.1 `cppcheck` on `tcc`

First, extract `tcc` with:

```
tar -xf tcc-0.9.27.tar.bz2
```

Then, run `cppcheck` on `tcc`. This can be accomplished with:

```
cppcheck --suppress=integerOverflow --output-file=<file> -j 4 *.c
```

Note: the `j` flag controls how many threads are used to run the check. You may wish to increase this.

4.1.2 `cppcheck` on `tcas`

Next, run `cppcheck` on `tcas5.c`. This time, run it twice:

1. Once as
`cppcheck --output-file=<file> tcas5.c`
2. Once as
`cppcheck --output-file=<file> --bug-hunting tcas5.c`

4.1.3 `infer` on `tcc`

1. Since `infer` works by interacting with the build process, you must first generate the Makefile by running `./configure`.
2. Run `infer` with
`infer run -j 1 --reactive -- make`
Note: We use `j=1` to make `infer` run on 1 cpu. This is important because in the report section we ask for you to reference the issues by number and if `infer` uses more than one CPU the order can change.
3. Run `infer explore --html` in the TCC directory to generate a full report, which you can then view by opening `files/hw4/tcc-0.9.27/infer-out/report.html/index.html` in a browser.

4.2 Write a report.

The report must be under 4 pages in `ieee-conference` format¹ and include:

1. Analysis of `tcas5.c`:
 - (a) Create a table containing each fault from both runs of `cpp` on `tcas` from subsection 4.1.2.
 - (b) For each item in the table, determine whether it is certainly a fault, potentially a fault, or a false positive. Justify your answer through your own analysis of the method in which the fault is reported. (*The analysis should be 1-4 sentences for each item.*)
2. Analysis of `tcc`:
 - (a) Create a table with each fault reported by `cppcheck` and `infer` on `tcc` when they are run with the flags given in subsection 4.1.1 and subsection 4.1.3, but excluding `infer`'s `UNINITIALIZED_VALUE` and `cppcheck`'s `note` reports. The table should include one row for each finding with [file name, line number, and the fault type reported], and be sorted by file name and then line number (do not include the code itself).
 - (b) Using the html report from subsection 4.1.3, for the four issue traces listed below, examine the traces produced by the analysis and, for each trace, explain if it is: certainly a fault, potentially a fault, or a false positive and give a brief (1-4 sentences each) explanation of your reasoning. For each of the four cases, only consider the code presented in the trace when making your determination. If `infer` references a snippet of a function, you are allowed to examine the rest of the function for more context.
 - i. Issue 1: `libtcc.c:174 Null Dereference pointer 'p'`
 - ii. Issue 8: `libtcc.c:441 Uninitialized Value`
 - iii. Issue 51: `tccelf.c:2604: error: Null Dereference pointer data`
 - iv. Issue 62: `tccgen.c:4100: error: Null Dereference pointer type->ref`
 - (c) Using the table and the trace analyses, discuss how likely the reported items are to be faults and what trends in types of reported items that you observe.
3. Limitations:

Review the flags/options for each tool. For the issues identified above, determine if the issue is a fundamental limitation of the approach or if it could be resolved through the use of existing flags.

¹Templates are available at <https://www.ieee.org/conferences/publishing/templates.html> or <https://www.overleaf.com/latex/templates/ieee-conference-template/grfzhnncsfqn>.

4. Summary:

A summary of the performance of both tools in terms of: issues found, potential severity, false positives, false negatives, etc.

5 Submission

5.1 Report

Submit your report through Canvas.

5.2 Code

There is no code submission for this assignment.